

Aula 9: Modelos de Sequência e Redes Recorrentes

Modelos Probabilísticos, RNNs, Gradientes e Modelos de Espaço de Estados

Eliezer de Souza da Silva
sereliezer.github.io
eliezer.silva@ufc.br

Mestrado e Doutorado em Ciência da Computação (MDCC)
Universidade Federal do Ceará

Novembro de 2025

Roteiro da Aula

- 1 Motivação e Dados Sequenciais
- 2 Modelagem Probabilística Clássica
- 3 Modelos de Espaço de Estados (HMMs)
- 4 Redes Neurais para Sequências
- 5 Redes Recorrentes (RNN)
- 6 Expressividade e Poder Computacional
- 7 Gradientes e Instabilidade
- 8 Modelos com Gates: LSTM e GRU
- 9 Arquiteturas Seq2Seq
- 10 Encerramento

Por que Estudar Modelos de Sequência? I

O Desafio

Muitos fenômenos do mundo são inerentemente **sequenciais** e de **comprimento variável**. Nossa meta é modelar a distribuição de probabilidade sobre essas sequências.

Por que Estudar Modelos de Sequência? II

Exemplos de Aplicações

- **Modelagem de Linguagem:** Prever a próxima palavra em um texto (ex: autocompletar).
- **Tradução Automática:** Mapear uma sequência de palavras de um idioma para outro.
- **Reconhecimento de Voz:** Transcrever uma sequência de áudio para texto.
- **Geração de Legendas:** Descrever uma imagem com uma sequência de palavras.
- **Análise de Séries Temporais:** Prever valores futuros em dados financeiros ou de sensores.

Por que Estudar Modelos de Sequência? III

Objetivo Comum

Em todos os casos, buscamos um modelo $p(x_1, \dots, x_N)$ que atribua alta probabilidade a sequências plausíveis e baixa a sequências implausíveis.

A Regra da Produto da Probabilidade I

Decomposição

Qualquer distribuição de probabilidade conjunta sobre uma sequência $x_1, \dots, x_T$ pode ser decomposta em um produto de probabilidades condicionais:

$$\begin{aligned} p(x_1, \dots, x_N) &= p(x_1) \cdot p(x_2|x_1) \cdot \dots \cdot p(x_N|x_1, \dots, x_{N-1}) \\ &= \prod_{i=1}^T p(x_i|x_{1:i-1}) \\ &= \prod_{i=1}^T p(x_i|\mathcal{H}_i) \end{aligned}$$

A Regra da Produto da Probabilidade II

- **O Problema:** O histórico $\mathcal{H}_t = x_{1:t-1} = \{x_1, \dots, x_{t-1}\}$ pode ser arbitrariamente longo. É impraticável estimar $p(x_i | \mathcal{H}_t)$ para todos os históricos possíveis.
- **A Solução Clássica:** Fazer uma suposição de simplificação.

A Regra da Produto da Probabilidade III

Suposição de Markov de Ordem $n - 1$ (Modelos N-gram)

O futuro é independente do passado distante, dado o passado recente $\mathcal{H}_t = \mathcal{H}_t(\delta) = \{x_{t-\delta+1}, \dots, x_{t-1}\}$:

$$p(x_t | x_{1:t-1}) \approx p(x_t | x_{t-\delta+1:t-1})$$

- $\delta = 1$: Unigrama (independência)
- $\delta = 2$: Bigrama (depende da valor anterior)
- $\delta = 3$: Trigrama (depende dos dois valores anteriores)
- $\delta = n$: ngrama (depende das n valores anteriores)

Um Primeiro Modelo (Ingênuo): Contar Sequências I

A Ideia Mais Simples

Se o nosso universo de sequências fosse pequeno, poderíamos estimar a probabilidade de uma sequência x simplesmente pela sua frequência no corpus de treinamento:

$$p(x) \propto \text{count}(x)$$

Isso significa que a plausibilidade de uma sequência é diretamente proporcional ao número de vezes que a vimos.

Um Primeiro Modelo (Ingênuo): Contar Sequências II

O Problema Fatal: Generalização Zero

O que acontece se uma nova sequência $\bar{x}$ (nossos dados de teste) nunca apareceu *exatamente* daquela forma nos dados de treinamento?

- $\text{count}(\bar{x}) = 0 \implies p(\bar{x}) = 0$
- A perplexidade se torna infinita, e o modelo é inútil para qualquer sequência nova.

Este modelo memoriza, mas não generaliza. É por isso que a decomposição da regra da cadeia, mesmo com a simplificação de Markov, é essencial: ela nos permite compor probabilidades de pedaços menores (n-gramas) que vimos, para avaliar sequências inteiras que nunca vimos.

Estimação por Máxima Verossimilhança (MLE) I

Como estimar $p(v|h)$?

A estimativa de máxima verossimilhança (MLE) para um modelo n-gram é simplesmente contar e normalizar.

$$\theta_{v|h}^* = p_{MLE}(x_i = v | x_{i-n+1:i-1} = h) = \frac{\text{count}(h, v)}{\text{count}(h)}$$

onde h é o histórico de $n - 1$ palavras e v é a palavra seguinte.

Estimação por Máxima Verossimilhança (MLE) II

Exemplo de Bigrama ($n = 2$)

Para a frase “o gato sentou no tapete”:

$$p(\text{sentou}|\text{gato}) = \frac{\text{count}(\text{"gato sentou"})}{\text{count}(\text{"gato"})}$$

Essas contagens são feitas sobre um grande corpus de treinamento.

Avaliação: Perplexidade I

O que é Perplexidade (PP)?

É uma medida intrínseca de quão bem um modelo de linguagem prediz uma amostra de teste. É o inverso da probabilidade geométrica média por palavra.

$$PP(\bar{x}) = \left(\frac{1}{p(\bar{x}_1, \dots, \bar{x}_{\bar{N}})} \right)^{1/\bar{N}} = 2^{-\frac{1}{\bar{N}} \sum_{i=1}^{\bar{N}} \log_2 p(\bar{x}_i | \bar{x}_{<i})}$$

- Corresponde à exponenciação da entropia cruzada.
- **Interpretação:** Pode ser visto como o "tamanho efetivo do vocabulário" que o modelo precisa escolher a cada passo.

Avaliação: Perplexidade II

- **Objetivo:** Minimizar a perplexidade. Um modelo perfeito teria $PP=1$. Um modelo que atribui probabilidade uniforme de $1/V$ a tudo teria $PP=V$.

Limitações dos Modelos N-gram I

Problemas Fundamentais

- **Esparsidade:** Muitos n-gramas válidos nunca aparecem nos dados de treino, recebendo probabilidade zero. Ex: "gato roxo sentou". Isso leva a $PP = \infty$.

Limitações dos Modelos N-gram II

Problema Conceitual

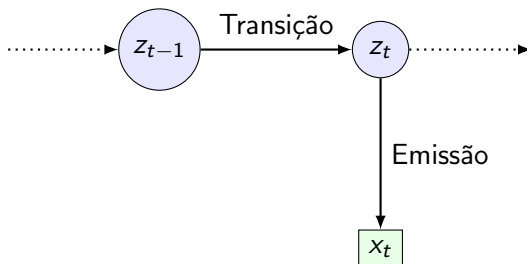
- **Generalização Semântica:** O modelo não compartilha conhecimento entre palavras similares. Se ele aprende sobre "gato", não sabe nada sobre "felino".
- **Dependências de Longo Prazo:** A janela de contexto fixa $(n - 1)$ impede a captura de dependências que vão além dessa janela.

Precisamos de uma forma de representar o histórico de maneira mais compacta e generalizável.

A Próxima Evolução: Modelos com Estado Latente I

A Ideia Central

Em vez de condicionar nos últimos $n - 1$ símbolos observados, vamos postular a existência de um **estado latente (ou oculto)** z_t que resume toda a informação necessária do passado.



A Próxima Evolução: Modelos com Estado Latente II

Duas Suposições Fundamentais

- 1 Suposição de Markov para os Estados:** O estado atual z_t depende apenas do estado anterior z_{t-1} .

$$p(z_t | z_1, \dots, z_{t-1}, x_1, \dots, x_{t-1}) = p(z_t | z_{t-1})$$

- 2 Suposição de Independência das Observações:** A observação atual x_t depende apenas do estado atual z_t .

$$p(x_t | z_1, \dots, z_t, x_1, \dots, x_{t-1}) = p(x_t | z_t)$$

Este é o **Modelo Oculto de Markov (HMM)**.

HMMs: Definição Formal I

Um HMM é definido por um conjunto de probabilidades:

- **Espaço de Estados $\mathcal{Z}$:** Um conjunto de K estados ocultos, $\{s_1, \dots, s_K\}$.
- **Espaço de Observações $\mathcal{X}$:** Um conjunto de possíveis observações.
- **Probabilidades de Transição A :** Uma matriz $K \times K$ onde $A_{ij} = p(z_t = s_j | z_{t-1} = s_i)$.
- **Probabilidades de Emissão B :** Descrevem $p(x_t | z_t = s_j)$. A forma de B depende se as observações são discretas ou contínuas.
- **Distribuição Inicial π :** Um vetor de tamanho K onde $\pi_i = p(z_1 = s_i)$.

HMMs: Definição Formal II

Verossimilhança Conjunta

A probabilidade de uma sequência de estados e observações é dada por:

$$p(x_{1:T}, z_{1:T}) = p(z_1) \prod_{t=2}^T p(z_t | z_{t-1}) \prod_{t=1}^T p(x_t | z_t)$$

HMMs Discretos vs. Contínuos I

Modelo Discreto

Se tanto os estados quanto as observações são discretos, as emissões são uma matriz B de tamanho $K \times |\mathcal{X}|$.

- **Exemplo:** Dinâmica do tempo ($z_t \in (\text{Sol}, \text{Chuva})$) e número de sorvetes vendidos $x_t \in (\text{Poucos}, \text{Muitos})$.
- $p(z_t = \text{Sol} | z_{t-1} = \text{Chuva})$, $p(x_t = \text{Muitos} | z_t = \text{Sol})$, etc.

HMMs Discretos vs. Contínuos II

Modelo Contínuo

Se as observações $x_t \in \mathbb{R}^D$ são contínuas, a emissão é uma densidade de probabilidade, frequentemente uma Gaussiana:

$$p(x_t | z_t = s_k) = \mathcal{N}(x_t; \mu_k, \Sigma_k)$$

Cada estado s_k tem sua própria média μ_k e covariância Σ_k .

HMMs Discretos vs. Contínuos III

HMMs com Dinâmica Linear-Gaussiana

Um caso especial muito importante de HMMs contínuos é quando:

- 1 A dinâmica de transição dos estados é **linear**.
- 2 A função de emissão é **linear**.
- 3 O ruído do processo e da observação é **Gaussiano**.

Isso nos leva a um **Sistema Dinâmico Linear**, e a tarefa de inferir o estado oculto z_t a partir das observações $x_{1:t}$ é resolvida pelo famoso **Filtro de Kalman**.

As Equações do Filtro de Kalman I

Definição do Sistema Dinâmico Linear

- **Equação de Transição:** Como o estado $z_t \in \mathbb{R}^k$ evolui.

$$z_t = Az_{t-1} + w_{t-1}, \quad w_{t-1} \sim \mathcal{N}(0, Q)$$

- **Equação de Observação:** Como a observação $x_t \in \mathbb{R}^d$ é gerada.

$$x_t = Cz_t + v_t, \quad v_t \sim \mathcal{N}(0, R)$$

Onde A é a matriz de transição, C é a matriz de observação, e Q, R são as matrizes de covariância do ruído do processo e da observação, respectivamente.

As Equações do Filtro de Kalman II

O Algoritmo Recursivo: Prever e Corrigir

O filtro mantém uma estimativa da distribuição do estado

$$p(z_t | x_{1:t}) = \mathcal{N}(\hat{z}_t, P_t).$$

1 Passo de Predição (a priori):

$$\hat{z}_{t|t-1} = A\hat{z}_{t-1|t-1} \quad (\text{Estado previsto})$$

$$P_{t|t-1} = AP_{t-1|t-1}A^\top + Q \quad (\text{Incerteza prevista})$$

2 Passo de Atualização (a posteriori):

$$K_t = P_{t|t-1}C^\top(CP_{t|t-1}C^\top + R)^{-1} \quad (\text{Ganho de Kalman})$$

$$\hat{z}_{t|t} = \hat{z}_{t|t-1} + K_t(x_t - C\hat{z}_{t|t-1}) \quad (\text{Estado atualizado})$$

$$P_{t|t} = (I - K_tC)P_{t|t-1} \quad (\text{Incerteza atualizada})$$

A Ponte para as Redes Recorrentes I

O que aprendemos com os HMMs?

A ideia de um **estado interno** z_t que resume o passado é extremamente poderosa. No entanto, a dinâmica dos HMMs e Filtros de Kalman é restrita (tabular ou linear).

A Questão-Chave

E se pudéssemos tornar a função de atualização do estado arbitrariamente poderosa e não-linear, capaz de capturar dinâmicas complexas como as da linguagem humana?

A Ponte para as Redes Recorrentes II

Filtro de Kalman (Dinâmica Linear)

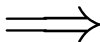
$$z_t = Az_{t-1} + Bx_t + \text{ruído}$$

Poder expressivo limitado, mas
com inferência exata e ótima.

Rede Neural Recorrente (RNN) (Dinâmica Não-Linear)

$$h_t = f_{\theta}(h_{t-1}, x_t)$$

Poder expressivo universal. f_{θ} é
uma rede neural que **aprende** a
dinâmica a partir dos dados.



A Ponte para as Redes Recorrentes III

A Ideia da RNN

Uma RNN generaliza os modelos de espaço de estados. O "estado oculto" h_t é um vetor em $\mathbb{R}^d$, e a função de transição é uma rede neural parametrizada, que aprendemos de ponta a ponta.

De N-gramas para Modelos Neurais I

A Ideia Central

Vamos substituir as tabelas de contagem por uma função neural que aprende a resumir o histórico.

$$p(x_t | x_{1:t-1}) = f(h_t) \quad \text{onde} \quad h_t = g(x_{1:t-1})$$

- $g(\cdot)$ é um **encoder** que mapeia o histórico de comprimento variável para um vetor de estado de tamanho fixo $h_t \in \mathbb{R}^d$.
- $f(\cdot)$ é um **decoder** (geralmente uma camada linear + softmax) que mapeia o estado para uma distribuição de probabilidade sobre o vocabulário.

De N-gramas para Modelos Neurais II

Vantagens

- 1 O estado h_t pode, em teoria, capturar toda a informação do histórico.
- 2 A estimação é feita via gradiente, compartilhando parâmetros e permitindo generalização.

Representando Palavras: Embeddings I

De one-hot para vetores densos

Primeiro, representamos cada palavra v_i do vocabulário $\mathcal{V}$ como um vetor "one-hot" $e_i \in \mathbb{R}^{|\mathcal{V}|}$.

$$e_i = [0, \dots, 1, \dots, 0]^\top$$

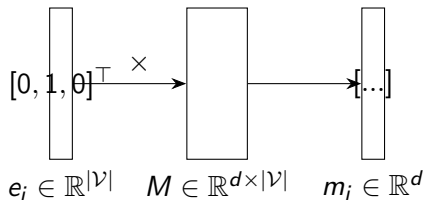
Isso é esparsos e não captura similaridade.

Representando Palavras: Embeddings II

Matriz de Embedding

Usamos uma matriz de embedding $M \in \mathbb{R}^{d \times |\mathcal{V}|}$ que é **aprendida** junto com o resto do modelo. Cada palavra v_i é mapeada para um vetor denso $m_i \in \mathbb{R}^d$.

$$m_i = Me_i \quad (\text{na prática, uma busca na } i\text{-ésima coluna de } M)$$



Representando Palavras: Embeddings III

Palavras com significados similares (e.g., "gato", "felino") terão vetores m_i próximos no espaço de embedding.

A Ideia da Recorrência I

Estado Recorrente

Uma Rede Neural Recorrente (RNN) define o estado oculto h_t como uma função do estado anterior h_{t-1} e da entrada atual x_t .

$$h_t = f(h_{t-1}, x_t; \theta)$$

A forma mais simples (Elman RNN) é:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

$$\hat{y}_t = \text{softmax}(W_{hy}h_t + b_y)$$

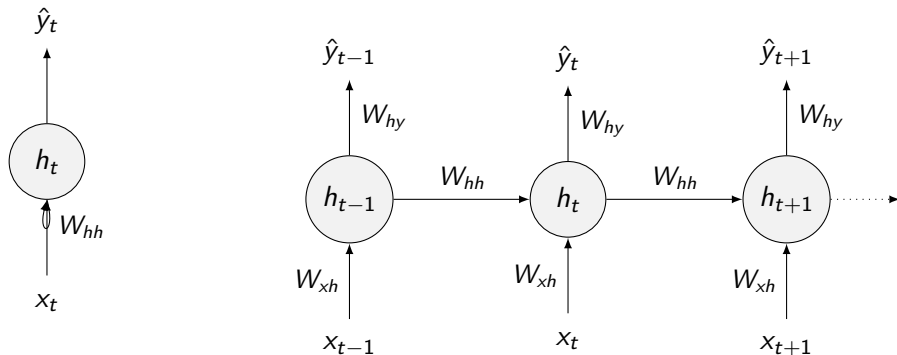
A Ideia da Recorrência II

Propriedade Fundamental

Os mesmos pesos (W_{hh} , W_{xh} , W_{hy}) e vieses (b_h , b_y) são **reutilizados em cada passo de tempo**. A rede tem uma forma de "memória" que é atualizada a cada passo, e os parâmetros que governam essa atualização são compartilhados ao longo de toda a sequência.

Visualização: Desdobramento no Tempo (Unrolling) I

Uma RNN pode ser vista como uma rede feed-forward muito profunda, onde cada camada corresponde a um passo de tempo e os pesos são compartilhados entre as camadas.



Poder Computacional: RNNs são Turing-Completas I

Resultado Teórico (Siegelmann & Sontag, 1995)

Uma Rede Neural Recorrente com um número suficiente de neurônios, pesos racionais e ativações lineares por partes (como a sigmoide saturada) pode simular qualquer Máquina de Turing.

Poder Computacional: RNNs são Turing-Completas II

Implicações

- **Universalidade:** Em teoria, uma RNN é um computador universal. Ela pode executar qualquer algoritmo que um computador convencional pode.
- **Capacidade Ilimitada:** Isso significa que as RNNs têm a capacidade teórica de aprender qualquer padrão computável em dados sequenciais, incluindo gramáticas complexas e dependências de longuíssimo prazo.

Poder Computacional: RNNs são Turing-Completas III

Caveats

Este é um resultado sobre a **expressividade** (o que a rede *pode* representar), não sobre a **aprendizagem** (o que pode ser aprendido eficientemente com gradiente descendente). A teoria requer:

- Precisão infinita nos estados e pesos.
- Potencialmente um número muito grande de neurônios e tempo de processamento.

Na prática, os problemas de otimização (gradientes) limitam o que podemos aprender.

Perspectiva de Sistemas Dinâmicos: RNNs e SDEs I

RNNs como Sistemas Dinâmicos Discretos

A equação de atualização de uma RNN pode ser reescrita como:

$$h_t = h_{t-1} + \underbrace{(\tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h) - h_{t-1})}_{\Delta h_t}$$

Isso se parece com uma discretização de Euler de uma **Equação Diferencial Ordinária (EDO)**:

$$\frac{dh(t)}{dt} = f(h(t), x(t))$$

Onde o estado $h(t)$ evolui continuamente no tempo, guiado pela entrada $x(t)$. A RNN aprende a dinâmica f .

Perspectiva de Sistemas Dinâmicos: RNNs e SDEs II

Adicionando Incerteza: Equações Diferenciais Estocásticas (SDEs)

Podemos modelar a incerteza ou o ruído nos dados sequenciais vendo a RNN como uma discretização de uma SDE:

$$dh_t = \underbrace{f(h_t, x_t)dt}_{\text{drift/determinístico}} + \underbrace{g(h_t, x_t)dW_t}_{\text{difusão/estocástico}}$$

- dW_t é um processo de Wiener (ruído branco Gaussiano).
- Essa visão conecta RNNs a campos como controle ótimo, filtragem de Kalman e física estatística.
- Inspira novas arquiteturas como as **Redes Neurais de EDO (Neural ODEs)**, que aprendem a dinâmica contínua f e usam solvers de EDO para fazer previsões.

Backpropagation Através do Tempo (BPTT) I

Como treinar uma RNN?

- 1 Desdobramos a rede no tempo para a sequência de treinamento.
- 2 Calculamos a perda total $J = \sum_{t=1}^N J_t$, onde J_t é a perda no tempo t (e.g., entropia cruzada entre $\hat{y}_t$ e y_t).
- 3 Calculamos o gradiente da perda total em relação a cada parâmetro θ : $\frac{\partial J}{\partial \theta} = \sum_{t=1}^N \frac{\partial J_t}{\partial \theta}$.
- 4 Atualizamos os pesos usando gradiente descendente.

Backpropagation Através do Tempo (BPTT) II

O Desafio

O gradiente $\frac{\partial J_t}{\partial \theta}$ em um tempo t depende dos estados de todos os tempos anteriores $k < t$. Isso leva a um caminho de retropropagação muito longo.

Derivação do Gradiente Recorrente I

Consideremos o gradiente da perda J_t em relação a um estado anterior h_k (com $k < t$). Pela regra da cadeia:

$$\frac{\partial J_t}{\partial h_k} = \frac{\partial J_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} = \frac{\partial J_t}{\partial h_t} \left(\prod_{i=k+1}^t \frac{\partial h_i}{\partial h_{i-1}} \right)$$

A derivada do estado (o Jacobiano) é:

$$\frac{\partial h_i}{\partial h_{i-1}} = \tanh'(\dots) \cdot W_{hh} = \text{diag}(1 - h_i^2) \cdot W_{hh}$$

O gradiente total é um produto de $t - k$ matrizes Jacobianas:

$$\frac{\partial J_t}{\partial h_k} = \frac{\partial J_t}{\partial h_t} \left(\prod_{i=k+1}^t \text{diag}(1 - h_i^2) W_{hh} \right)$$

Desaparecimento e Explosão de Gradientes I

O produto de Jacobianos $\prod_{i=k+1}^t \frac{\partial h_i}{\partial h_{i-1}}$ é a fonte do problema.

Desaparecimento de Gradientes (Vanishing)

Se as matrizes Jacobianas tiverem normas (valores singulares) consistentemente menores que 1, o produto decairá exponencialmente para zero.

$$0.9 \times 0.9 \times \cdots \times 0.9 \rightarrow 0$$

Consequência: A rede não consegue aprender dependências de longo prazo, pois o sinal de erro do futuro distante não chega ao passado.

Desaparecimento e Explosão de Gradientes II

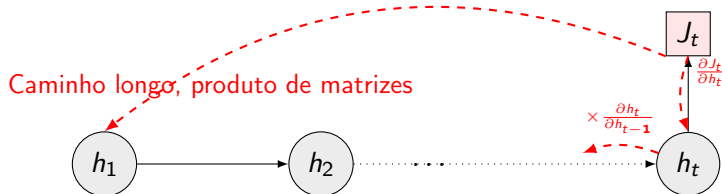
Explosão de Gradientes (Exploding)

Se as normas forem consistentemente maiores que 1, o produto crescerá exponencialmente.

$$1.1 \times 1.1 \times \cdots \times 1.1 \rightarrow \infty$$

Consequência: Os gradientes se tornam 'NaN' ou infinitos, desestabilizando completamente o treinamento.

Desaparecimento e Explosão de Gradientes III



O gradiente de J_t em relação aos parâmetros que influenciaram h_1 se torna muito pequeno ou muito grande.

Soluções Práticas I

■ Explosão de Gradientes:

- **Gradient Clipping:** Uma solução simples e eficaz. Se a norma do gradiente total, $||\nabla_{\theta} J||$, exceder um limiar, nós a reescalamos:

$$\text{if } ||\nabla_{\theta} J|| > \text{threshold, then } \nabla_{\theta} J \leftarrow \frac{\text{threshold}}{||\nabla_{\theta} J||} \nabla_{\theta} J$$

■ Desaparecimento de Gradientes:

- **Inicialização Cuidadosa:** Inicializar W_{hh} como uma matriz identidade ou ortogonal.
- **Funções de Ativação:** Usar ReLU pode ajudar, mas pode levar a estados que "morrem" (sempre zero).
- **Arquiteturas com Gates (LSTM, GRU):** A solução mais bem-sucedida e amplamente utilizada. Elas criam "atalhos" para o fluxo do gradiente.

LSTM: Long Short-Term Memory I

Proposta por Hochreiter & Schmidhuber (1997), a LSTM introduz um **estado de célula** (C_t) que atua como uma "via expressa" para a informação, permitindo que ela flua sem grandes alterações por muitos passos. O fluxo é controlado por três "portões"(gates).

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (\text{Forget Gate: O que esquecer da célula?})$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (\text{Input Gate: O que adicionar à célula?})$$

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C) \quad (\text{Candidato a novo estado})$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (\text{Atualização da célula})$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (\text{Output Gate: O que revelar?})$$

$$h_t = o_t \odot \tanh(C_t) \quad (\text{Novo estado oculto})$$

LSTM: Long Short-Term Memory II

onde σ é a função sigmoide e $\odot$ é a multiplicação elemento a elemento.

Gráfico Computacional da LSTM: Decomposição I

Entradas e Estado Interno

A célula LSTM opera em cada passo de tempo t recebendo o estado oculto anterior h_{t-1} e a entrada atual x_t . Ela mantém um **estado de célula** interno, C_{t-1} , que atua como sua memória de longo prazo. A computação é dividida em três estágios principais controlados por "gates".

Gráfico Computacional da LSTM: Decomposição II

1. Estágio de "Esquecimento"(Forget Gate Layer)

Primeiro, a célula decide qual informação do estado de célula anterior (C_{t-1}) deve ser descartada. A função sigmoide produz um número entre 0 ("esquecer completamente") e 1 ("manter completamente") para cada elemento do vetor C_{t-1} .

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Gráfico Computacional da LSTM: Decomposição III

2. Estágio de "Entrada/Armazenamento" (Input Gate Layer)

Em seguida, a célula decide quais novas informações serão armazenadas. Isso ocorre em duas partes:

- O "portão de entrada" (i_t) decide quais valores serão atualizados.

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

- Uma camada 'tanh' cria um vetor de novos valores candidatos, $\tilde{C}_t$, que poderiam ser adicionados ao estado.

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$$

O estado de célula é então atualizado: $C_t = (f_t \odot C_{t-1}) + (i_t \odot \tilde{C}_t)$

Gráfico Computacional da LSTM: Decomposição IV

3. Estágio de "Saída"(Output Gate Layer)

Finalmente, a célula decide qual será o próximo estado oculto h_t . O "portão de saída"(o_t) decide qual parte do estado de célula será revelada. O estado de célula é passado por uma 'tanh' e multiplicado pelo portão de saída.

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

Gráfico Computacional da LSTM: Decomposição V

Fluxo do Gradiente

A atualização do estado da célula, $C_t = f_t \odot C_{t-1} + \dots$, é a chave. A natureza **aditiva** da operação cria um caminho direto para o gradiente fluir através do tempo via C_{t-1} , combatendo o desaparecimento. Os gates aprendem a modular esse fluxo.

GRU: Gated Recurrent Unit I

Proposta por Cho et al. (2014), a GRU é uma versão simplificada da LSTM com menos parâmetros. Ela combina o estado de célula e o estado oculto e usa apenas dois gates.

$$\begin{aligned}
 z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z) && \text{(Update Gate: Quanto do novo estado)} \\
 r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r) && \text{(Reset Gate: Quanto do estado antigo)} \\
 \tilde{h}_t &= \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h) && \text{(Candidato a novo estado)} \\
 h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t && \text{(Novo estado oculto)}
 \end{aligned}$$

- O 'update gate' (z_t) controla a interpolação entre o estado antigo (h_{t-1}) e o novo candidato ($\tilde{h}_t$).

GRU: Gated Recurrent Unit II

- O 'reset gate' (r_t) decide o quanto do estado anterior é usado para calcular o novo candidato.
- Frequentemente tem desempenho similar à LSTM, mas é computacionalmente mais eficiente.

Arquitetura Encoder-Decoder I

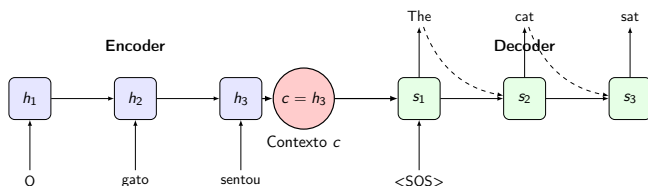
Para tarefas que mapeiam uma sequência de entrada para uma sequência de saída de comprimento diferente (e.g., tradução), usamos a arquitetura **Sequence-to-Sequence (Seq2Seq)**.

- **Encoder:** Uma RNN (ou LSTM/GRU) processa a sequência de entrada $(x_1, \dots, x_S)$ e a resume em um único vetor de contexto c . Geralmente, c é o último estado oculto do encoder, h_S .
- **Decoder:** Outra RNN, inicializada com o contexto c , gera a sequência de saída $(y_1, \dots, y_T)$ um token de cada vez.

O modelo aprende a probabilidade condicional:

$$p(y_1, \dots, y_T | x_1, \dots, x_S) = \prod_{t=1}^T p(y_t | y_{<t}, c)$$

Visualização da Arquitetura Seq2Seq I



Funcionamento do Decoder

- 1 O decoder recebe o contexto c e um token especial de início ($\langle \text{SOS} \rangle$).
- 2 Ele gera a primeira palavra da saída ('The').
- 3 Essa palavra gerada é então usada como entrada para o próximo passo, para gerar a segunda palavra ('cat'), e assim por diante, até um token de fim ($\langle \text{EOS} \rangle$) ser produzido.

Aplicações e Limitações do Seq2Seq I

Aplicações

- Tradução automática.
- Sumarização de documentos.
- Geração de respostas em chatbots.
- Geração de código a partir de descrições.

Limitação: O Gargalo do Contexto

Toda a informação da sequência de entrada precisa ser comprimida em um **único vetor de contexto** c .

Isso é um gargalo de informação, especialmente para sequências longas. A rede pode "esquecer" o início da sequência.

Aplicações e Limitações do Seq2Seq II

*Esta limitação motivou o desenvolvimento do **mecanismo de atenção**, que permite ao decoder olhar para diferentes partes da entrada a cada passo. Isso será o tópico da próxima aula e a base dos Transformers.*

Resumo da Aula I

- Vimos como modelar sequências, começando com a **regra da cadeia** e a simplificação dos **modelos n-gram**.
- Introduzimos as **Redes Neurais Recorrentes (RNNs)**, destacando seu poder teórico como **computadores universais (Turing-completos)** e sua conexão com **sistemas dinâmicos contínuos**.
- Analisamos o desafio prático da **instabilidade dos gradientes** (desaparecimento e explosão) no treinamento via BPTT.
- Exploramos as **LSTMs e GRUs** como soluções arquitetônicas que usam gates para controlar o fluxo de informação e gradientes, permitindo o aprendizado de dependências de longo prazo.

Resumo da Aula II

- Apresentamos a arquitetura **Encoder-Decoder (Seq2Seq)** para tarefas de mapeamento de sequência para sequência.

Leitura Adicional

Noah A. Smith (2025), *Introduction to Sequence Models*. Disponível no material do curso.

Hava T. Siegelmann and Eduardo D. Sontag (1995), *On the Computational Power of Neural Nets*. Journal of Computer and System Sciences.

Ian Goodfellow, Yoshua Bengio, and Aaron Courville (2016), *Deep Learning*, MIT Press.
Capítulo 10: "Sequence Modeling: Recurrent and Recursive Nets".